

一、BOM 操作知识点整理

1.1 浏览器对象模型 Browser Object Model

不存在浏览器对象模型（BOM）的官方标准。

现代的浏览器已经（几乎）实现了 JavaScript 交互相同的方法和属性，因此它经常作为 BOM 的方法和属性被提到。

所有浏览器都支持 *window* 对象。它代表浏览器的窗口。

所有全局 JavaScript 对象，函数和变量自动成为 *window* 对象的成员。

全局变量是 *window* 对象的属性。

全局函数是 *window* 对象的方法。

甚至（HTML DOM 的）*document* 对象也是 *window* 对象属性：

```
window.document.getElementById("header");
```

等同于：

```
document.getElementById("header");
```

1.1.1 获取窗口大小

window.innerWidth* 和 *window.innerHeight

获取浏览器窗口的视口宽度和高度（不包括工具栏、滚动条等）。

```
console.log(window.innerWidth); // 视口宽度  
console.log(window.innerHeight); // 视口高度
```

window.outerWidth* 和 *window.outerHeight

获取整个浏览器窗口的宽度和高度（包括工具栏、滚动条等）。

```
console.log(window.outerWidth); // 窗口总宽度  
console.log(window.outerHeight); // 窗口总高度
```

1.1.2 滚动操作

`window.scrollTo(x, y)`
滚动到指定的坐标。

```
window.scrollTo(0, 1000); // 滚动到垂直方向 1000px
```

`window.scrollBy(x, y)`
相对于当前位置滚动。

```
window.scrollBy(0, 200); // 向下滚动 200px
```

1.1.3 获取滚动位置

`window.scrollX` 和 `window.scrollY`
获取页面水平和垂直方向的滚动距离。

```
console.log(window.scrollX); // 水平滚动距离  
console.log(window.scrollY); // 垂直滚动距离
```

1.1.4 页面加载事件

1.1.5 `window.onload`

当页面完全加载（包括图片、样式表等资源）时触发。

```
window.onload = function () {  
    console.log("Page fully loaded.");  
};
```

1.1.6 盒子距离浏览器的距离

在 JavaScript 中，可以通过以下属性获取一个元素相对于浏览器窗口或文档的位置：

使用 `getBoundingClientRect()`

- **定义：**
 - 返回元素的大小及其相对于视口（浏览器窗口）的位置。
- **返回值：**
 - 包含 `top`、`right`、`bottom`、`left`、`width` 和 `height` 属性。

1.1.6.1 示例代码

```
const box = document.querySelector('.box');
const rect = box.getBoundingClientRect();

console.log(rect.top);    // 元素顶部距离视口顶部的距离
console.log(rect.left);  // 元素左侧距离视口左侧的距离
console.log(rect.width); // 元素的宽度
console.log(rect.height); // 元素的高度
```

- 注意：

- `getBoundingClientRect()` 返回的是相对于视口的位置。如果页面有滚动条，需要加上 `window.scrollY` 或 `window.scrollX` 来计算相对于文档的位置。

1.1.6.2 计算相对于文档的位置

```
const box = document.querySelector('.box');
const rect = box.getBoundingClientRect();

const topRelativeToDocument = rect.top + window.scrollY;
const leftRelativeToDocument = rect.left + window.scrollX;

console.log(topRelativeToDocument); // 元素顶部距离文档顶部的距离
console.log(leftRelativeToDocument); // 元素左侧距离文档左侧的距离
```

二、监听事件补充

2.1 窗口大小变化

- 事件名称： `resize`
- 触发时机：当浏览器窗口的大小发生变化时触发。
- 示例代码：

```
window.addEventListener('resize', () => {
    console.log('窗口宽度:', window.innerWidth);
    console.log('窗口高度:', window.innerHeight);
});
```

实践案例：隐藏目录栏



2.2 页面滚动

- **事件名称:** scroll
- **触发时机:** 当用户滚动页面时触发。
- **示例代码:**

```
window.addEventListener('scroll', () => {
  console.log('垂直滚动距离:', window.scrollY);
  console.log('水平滚动距离:', window.scrollX);
});
```

2.3 鼠标事件

- **作用:** 监听鼠标移动
- **事件名称:** mousemove
- **触发时机:** 当鼠标在dom上移动时触发。
- **示例代码:**

```
document.addEventListener('mousemove', (e) => {
  console.log('鼠标 X 坐标:', e.clientX); // 相对于视口
  的水平位置
  console.log('鼠标 Y 坐标:', e.clientY); // 相对于视口
  的垂直位置
});
```

- **作用:** 监听鼠标移入
- **事件名称:** mouseenter
- **触发时机:** 当鼠标移入dom时触发。
- **示例代码:**

```
document.addEventListener('mouseenter', () => {
  console.log('鼠标移入了');
});
```

- **作用:** 监听鼠标移出
- **事件名称:** mouseleave

- **触发时机**：当鼠标移出dom时触发。
- **示例代码**：

```
document.addEventListener('mouseleave', () => {  
    console.log('鼠标移出了');  
});
```

三、网络请求

1. 什么是AJAX？

- 例如：在浏览器网页中，使用 AJAX技术（XHR对象）发起获取省份列表数据的请求，服务器代码响应准备好的省份列表数据给前端，前端拿到数据数组以后，展示到网页

2. 什么是服务器？

- 可以暂时理解为提供数据的一台电脑

3. 怎么学 AJAX ？

- 这里使用一个第三方库叫 axios

3.1 引入axios库

```
# 引入网络js文件  
<script src="https://unpkg.com/axios/dist/axios.min.js">  
</script>  
  
# 或  
  
# 引入本地js文件  
<script src="./axios.min.js"></script>
```

3.2 使用axios

```
axios({  
    url: '目标资源地址'  
}).then((result) => {  
    // 对服务器返回的数据做后续处理  
})
```

免费测试接口API文档：<https://api.apipopen.top/swagger/index.html>

好看视频接口：<https://api.apipopen.top/api/getHaoKanVideo>

3.3 GET 和 POST 请求

3.3.1 GET 请求

- **定义：**从服务器获取数据。
- **示例代码：**

```
// 获取用户数据
axios({
  url: 'https://jsonplaceholder.typicode.com/posts',
  method: 'get',
}) // .then()可以获取到请求成功时服务器返回的数据
.then(response => {
  console.log('用户列表:', response.data);
});
```

- **params参数：**指定的数据

```
axios({
  url: 'https://jsonplaceholder.typicode.com/posts',
  method: 'get',
  // 加入params属性
  params: {
    // 只获取userId为2的数据
    userId: 2
  },
})
.then(response => {
  console.log('用户列表:', response.data);
});
```

3.3.2 POST 请求

- **定义：**向服务器提交数据。
- **示例代码：**

```
axios({
  url: 'https://jsonplaceholder.typicode.com/posts',
  method: 'post',
  data: {
    userId: 666,
    id: 666,
    title: 'viking的第一个数据',
    body: '这是一个测试内容'
  },
})
```

3.3.3 错误处理

- 使用 `catch` 捕获错误:

```
axios({
  // ...请求选项
}).then(result => {
  // 处理成功数据
}).catch(error => {
  // 处理失败错误
})
```

3.3.4 状态码

状态码	描述
1xx	信息
2xx	成功
3xx	重定向消息
4xx	客户端错误
5xx	服务端错误

3.4 接口文档

1. 接口文档：描述接口的文章（一般是后端工程师，编写和提供）
2. 接口：指的使用 AJAX 和 服务器通讯时，使用的 URL，请求方法，以及参数

接口 修改接口 运行 Mock			
请求参数			
Body 参数 application/x-www-form-urlencoded			
name	string	宠物名	必需
示例值: Hello Kitty			
status	string	宠物销售状态	必需
示例值: sold			
返回响应			
成功(201)			
HTTP 状态码: 201		内容格式: JSON	application/json
数据结构			
code	integer		必需
>= 0 <= 0			
data	object (Pet)		必需
id	integer <int64>	宠物ID编号	必需
>= 1			
category	object (Category)	分组	必需

3.5 实战案例：获取并展示用户数据

3.5.1 需求

- 从 <https://jsonplaceholder.typicode.com/users> 获取用户数据。
- 将用户的名字显示在页面上。

3.5.2 HTML

```
<ul id="userList"></ul>
```

3.5.3 JavaScript

```
// 获取 DOM 元素
const userList = document.getElementById('userList');

// 获取 DOM 元素
const userList = document.getElementById('userList');
// 使用 axios 获取用户数据
axios('https://jsonplaceholder.typicode.com/users')
  .then(response => {
    console.log(response.data);

    response.data.forEach(user => {
      const li = document.createElement('li');
      li.textContent = user.name;
      userList.appendChild(li);
    });
  })
  .catch(error => {
    console.error('请求失败:', error);
  });
```